

CONTROL AND OBSERVABILITY

Rewind file changes with checkpointing

Track file changes during agent sessions and restore files to any previous state

File checkpointing tracks file modifications made through the Write, Edit, and NotebookEdit tools during an agent session, allowing you to rewind files to any previous state. Want to try it out? Jump to the [interactive example](#).

With checkpointing, you can:

- **Undo unwanted changes** by restoring files to a known good state
- **Explore alternatives** by restoring to a checkpoint and trying a different approach
- **Recover from errors** when the agent makes incorrect modifications

 Only changes made through the Write, Edit, and NotebookEdit tools are tracked. Changes made through Bash commands (like `echo > file.txt` or `sed -i`) are not captured by the checkpoint system.

How checkpointing works

When you enable file checkpointing, the SDK creates backups of files before modifying them through the Write, Edit, or NotebookEdit tools. User messages in the response stream include a checkpoint UUID that you can use as a restore point.

Checkpoint works with these built-in tools that the agent uses to modify files:

Tool	Description
Write	Creates a new file or overwrites an existing file with new content
Edit	Makes targeted edits to specific parts of an existing file
NotebookEdit	Modifies cells in Jupyter notebooks (<code>.ipynb</code> files)

⚠ File rewinding restores files on disk to a previous state. It does not rewind the conversation itself. The conversation history and context remain intact after calling `rewindFiles()` (TypeScript) or `rewind_files()` (Python).

The checkpoint system tracks:

- Files created during the session
- Files modified during the session
- The original content of modified files

When you rewind to a checkpoint, created files are deleted and modified files are restored to their content at that point.

Implement checkpointing

To use file checkpointing, enable it in your options, capture checkpoint UUIDs from the response stream, then call `rewindFiles()` (TypeScript) or `rewind_files()` (Python) when you need to restore.

The following example shows the complete flow: enable checkpointing, capture the checkpoint UUID and session ID from the response stream, then resume the session later to rewind files. Each step is explained in detail below.

```

import asyncio
from claude_agent_sdk import (
    ClaudeSDKClient,
    ClaudeAgentOptions,
    UserMessage,
    ResultMessage,
)

async def main():
    # Step 1: Enable checkpointing
    options = ClaudeAgentOptions(
        enable_file_checkpointing=True,
        permission_mode="acceptEdits", # Auto-accept file edits without
prompting
        extra_args={
            "replay-user-messages": None
        }, # Required to receive checkpoint UUIDs in the response stream
    )

    checkpoint_id = None
    session_id = None

    # Run the query and capture checkpoint UUID and session ID
    async with ClaudeSDKClient(options) as client:
        await client.query("Refactor the authentication module")

    # Step 2: Capture checkpoint UUID from the first user message
    async for message in client.receive_response():
        if isinstance(message, UserMessage) and message.uuid and not
checkpoint_id:
            checkpoint_id = message.uuid
        if isinstance(message, ResultMessage) and not session_id:
            session_id = message.session_id

    # Step 3: Later, rewind by resuming the session with an empty prompt
    if checkpoint_id and session_id:
        async with ClaudeSDKClient(
            ClaudeAgentOptions(enable_file_checkpointing=True,
resume=session_id)
        ) as client:
            await client.query("") # Empty prompt to open the connection
            async for message in client.receive_response():
                await client.rewind_files(checkpoint_id)

```

```
        break
    print(f"Rewound to checkpoint: {checkpoint_id}")
```

```
asyncio.run(main())
```

1 Enable checkpointing

Configure your SDK options to enable checkpointing and receive checkpoint UUIDs:

Option	Python	TypeScript	Description
Enable checkpointing	<code>enable_file_checkpointing=True</code>	<code>enableFileCheckpointing: true</code>	Tracks file changes for rewinding
Receive checkpoint UUIDs	<code>extra_args={"replay-user-messages": None}</code>	<code>extraArgs: { 'replay-user-messages': null }</code>	Required to get user message UUIDs in the stream

```
options = ClaudeAgentOptions(
    enable_file_checkpointing=True,
    permission_mode="acceptEdits",
    extra_args={"replay-user-messages": None},
)

async with ClaudeSDKClient(options) as client:
    await client.query("Refactor the authentication module")
```

2 Capture checkpoint UUID and session ID

With the `replay-user-messages` option set (shown above), each user message in the response stream has a UUID that serves as a checkpoint.

For most use cases, capture the first user message UUID (`message.uuid`); rewinding to it restores all files to their original state. To store multiple checkpoints and rewind to intermediate states, see [Multiple restore points](#).

Capturing the session ID (`message.session_id`) is optional; you only need it if you want to rewind later, after the stream completes. If you're calling `rewindFiles()` immediately while still processing messages (as the example in [Checkpoint before risky operations](#) does), you

can skip capturing the session ID.

```
checkpoint_id = None
session_id = None

async for message in client.receive_response():
    # Update checkpoint on each user message (keeps the latest)
    if isinstance(message, UserMessage) and message.uuid:
        checkpoint_id = message.uuid
    # Capture session ID from the result message
    if isinstance(message, ResultMessage):
        session_id = message.session_id
```

3 Rewind files

To rewind after the stream completes, resume the session with an empty prompt and call `rewind_files()` (Python) or `rewindFiles()` (TypeScript) with your checkpoint UUID. You can also rewind during the stream; see [Checkpoint before risky operations](#) for that pattern.

```
async with ClaudeSDKClient(
    ClaudeAgentOptions(enable_file_checkpointing=True, resume=session_id)
) as client:
    await client.query("") # Empty prompt to open the connection
    async for message in client.receive_response():
        await client.rewind_files(checkpoint_id)
        break
```

If you capture the session ID and checkpoint ID, you can also rewind from the CLI:

```
claude -p --resume <session-id> --rewind-files <checkpoint-uuid>
```

Common patterns

These patterns show different ways to capture and use checkpoint UUIDs depending on your use

case.

Checkpoint before risky operations

This pattern keeps only the most recent checkpoint UUID, updating it before each agent turn. If something goes wrong during processing, you can immediately rewind to the last safe state and break out of the loop.

```
import asyncio
from claude_agent_sdk import ClaudeSDKClient, ClaudeAgentOptions, UserMessage

async def main():
    options = ClaudeAgentOptions(
        enable_file_checkpointing=True,
        permission_mode="acceptEdits",
        extra_args={"replay-user-messages": None},
    )

    safe_checkpoint = None

    async with ClaudeSDKClient(options) as client:
        await client.query("Refactor the authentication module")

        async for message in client.receive_response():
            # Update checkpoint before each agent turn starts
            # This overwrites the previous checkpoint. Only keep the latest
            if isinstance(message, UserMessage) and message.uuid:
                safe_checkpoint = message.uuid

            # Decide when to revert based on your own logic
            # For example: error detection, validation failure, or user input
            if your_revert_condition and safe_checkpoint:
                await client.rewind_files(safe_checkpoint)
                # Exit the loop after rewinding, files are restored
                break

    asyncio.run(main())
```

Multiple restore points

If Claude makes changes across multiple turns, you might want to rewind to a specific point rather than all the way back. For example, if Claude refactors a file in turn one and adds tests in turn two, you might want to keep the refactor but undo the tests.

This pattern stores all checkpoint UUIDs in an array with metadata. After the session completes, you can rewind to any previous checkpoint:

```

import asyncio
from dataclasses import dataclass
from datetime import datetime
from claude_agent_sdk import (
    ClaudeSDKClient,
    ClaudeAgentOptions,
    UserMessage,
    ResultMessage,
)

# Store checkpoint metadata for better tracking
@dataclass
class Checkpoint:
    id: str
    description: str
    timestamp: datetime

async def main():
    options = ClaudeAgentOptions(
        enable_file_checkpointing=True,
        permission_mode="acceptEdits",
        extra_args={"replay-user-messages": None},
    )

    checkpoints = []
    session_id = None

    async with ClaudeSDKClient(options) as client:
        await client.query("Refactor the authentication module")

        async for message in client.receive_response():
            if isinstance(message, UserMessage) and message.uuid:
                checkpoints.append(
                    Checkpoint(
                        id=message.uuid,
                        description=f"After turn {len(checkpoints) + 1}",
                        timestamp=datetime.now(),
                    )
                )
            if isinstance(message, ResultMessage) and not session_id:
                session_id = message.session_id

```

```

# Later: rewind to any checkpoint by resuming the session
if checkpoints and session_id:
    target = checkpoints[0] # Pick any checkpoint
    async with ClaudeSDKClient(
        ClaudeAgentOptions(enable_file_checkpointing=True,
resume=session_id)
    ) as client:
        await client.query("") # Empty prompt to open the connection
        async for message in client.receive_response():
            await client.rewind_files(target.id)
            break
    print(f"Rewound to: {target.description}")

asyncio.run(main())

```

Try it out

This complete example creates a small utility file, has the agent add documentation comments, shows you the changes, then asks if you want to rewind.

Before you begin, make sure you have the **Claude Agent SDK installed**.

1 Create a test file

Create a new file called `utils.py` (Python) or `utils.ts` (TypeScript) and paste the following code:

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
def divide(a, b):  
    if b == 0:  
        raise ValueError("Cannot divide by zero")  
    return a / b
```

2 Run the interactive example

Create a new file called `try_checkpointing.py` (Python) or `try_checkpointing.ts` (TypeScript) in the same directory as your utility file, and paste the following code.

This script asks Claude to add doc comments to your utility file, then gives you the option to rewind and restore the original.

```

import asyncio
from claude_agent_sdk import (
    ClaudeSDKClient,
    ClaudeAgentOptions,
    UserMessage,
    ResultMessage,
)

async def main():
    # Configure the SDK with checkpointing enabled
    # - enable_file_checkpointing: Track file changes for rewinding
    # - permission_mode: Auto-accept file edits without prompting
    # - extra_args: Required to receive user message UUIDs in the stream
    options = ClaudeAgentOptions(
        enable_file_checkpointing=True,
        permission_mode="acceptEdits",
        extra_args={"replay-user-messages": None},
    )

    checkpoint_id = None # Store the user message UUID for rewinding
    session_id = None # Store the session ID for resuming

    print("Running agent to add doc comments to utils.py...\n")

    # Run the agent and capture checkpoint data from the response stream
    async with ClaudeSDKClient(options) as client:
        await client.query("Add doc comments to utils.py")

        async for message in client.receive_response():
            # Capture the first user message UUID - this is our restore
point
            if isinstance(message, UserMessage) and message.uuid and not
checkpoint_id:
                checkpoint_id = message.uuid
            # Capture the session ID so we can resume later
            if isinstance(message, ResultMessage):
                session_id = message.session_id

    print("Done! Open utils.py to see the added doc comments.\n")

    # Ask the user if they want to rewind the changes
    if checkpoint_id and session_id:
        response = input("Rewind to remove the doc comments? (y/n): ")

```

```

if response.lower() == "y":
    # Resume the session with an empty prompt, then rewind
    async with ClaudeSDKClient(
        ClaudeAgentOptions(enable_file_checkpointing=True,
resume=session_id)
    ) as client:
        await client.query("") # Empty prompt opens the
connection
        async for message in client.receive_response():
            await client.rewind_files(checkpoint_id) # Restore
files

            break

        print(
            "\n✓ File restored! Open utils.py to verify the doc
comments are gone."
        )
    else:
        print("\nKept the modified file.")

asyncio.run(main())

```

This example demonstrates the complete checkpointing workflow:

1. **Enable checkpointing:** configure the SDK with `enable_file_checkpointing=True` and `permission_mode="acceptEdits"` to auto-approve file edits
2. **Capture checkpoint data:** as the agent runs, store the first user message UUID (your restore point) and the session ID
3. **Prompt for rewind:** after the agent finishes, check your utility file to see the doc comments, then decide if you want to undo the changes
4. **Resume and rewind:** if yes, resume the session with an empty prompt and call `rewind_files()` to restore the original file

3

Run the example

Run the script from the same directory as your utility file.

💡 Open your utility file (`utils.py` or `utils.ts`) in your IDE or editor before running the script. You'll see the file update in real-time as the agent adds doc comments, then revert back to the original when you choose to rewind.

Python

```
python try_checkpointing.py
```

TypeScript

```
npx tsx try_checkpointing.ts
```

You'll see the agent add doc comments, then a prompt asking if you want to rewind. If you choose yes, the file is restored to its original state.

Limitations

File checkpointing has the following limitations:

Limitation	Description
Write/Edit/NotebookEdit tools only	Changes made through Bash commands are not tracked
Same session	Checkpoints are tied to the session that created them
File content only	Creating, moving, or deleting directories is not undone by rewinding
Local files	Remote or network files are not tracked

Troubleshooting

Checkpointing options not recognized

If `enableFileCheckpointing` or `rewindFiles()` isn't available, you may be on an older SDK version.

Solution: Update to the latest SDK version:

- **Python:** `pip install --upgrade claude-agent-sdk`
- **TypeScript:** `npm install @anthropic-ai/claude-agent-sdk@latest`

User messages don't have UUIDs

If `message.uuid` is `undefined` or missing, you're not receiving checkpoint UUIDs.

Cause: The `replay-user-messages` option isn't set.

Solution: Add `extra_args={"replay-user-messages": None}` (Python) or `extraArgs: { 'replay-user-messages': null }` (TypeScript) to your options.

"No file checkpoint found for message" error

This error occurs when the checkpoint data doesn't exist for the specified user message UUID.

Common causes:

- File checkpointing was not enabled on the original session (`enable_file_checkpointing` or `enableFileCheckpointing` was not set to `true`)
- The session wasn't properly completed before attempting to resume and rewind

Solution: Ensure `enable_file_checkpointing=True` (Python) or `enableFileCheckpointing: true` (TypeScript) was set on the original session, then use the pattern shown in the examples: capture the first user message UUID, complete the session fully, then resume with an empty prompt and call `rewindFiles()` once.

"ProcessTransport is not ready for writing" error

This error occurs when you call `rewindFiles()` or `rewind_files()` after you've finished iterating through the response. The connection to the CLI process closes when the loop completes.

Solution: Resume the session with an empty prompt, then call rewind on the new query:

```
# Resume session with empty prompt, then rewind
async with ClaudeSDKClient(
    ClaudeAgentOptions(enable_file_checkpointing=True, resume=session_id)
) as client:
    await client.query("")
    async for message in client.receive_response():
        await client.rewind_files(checkpoint_id)
    break
```

Next steps

- **Sessions**: learn how to resume sessions, which is required for rewinding after the stream completes. Covers session IDs, resuming conversations, and session forking.
- **Permissions**: configure which tools Claude can use and how file modifications are approved. Useful if you want more control over when edits happen.
- **TypeScript SDK reference**: complete API reference including all options for `query()` and the `rewindFiles()` method.
- **Python SDK reference**: complete API reference including all options for `ClaudeAgentOptions` and the `rewind_files()` method.